

移动数字证书(CA)互认技术标准
电子招标投标交易系统/交易工具接入指南

2024 年 7 月

目 录

1. 总体架构图	- 1 -
1.1. 软件部署及运维	- 1 -
1.2. 部署架构图	- 2 -
2. CA 互认共享组件部署	- 2 -
2.1. 硬件及环境准备	- 2 -
3. 交易系统/交易工具对接说明	- 3 -
3.1. 共享登录注册	- 3 -
3.2. 签名验签	- 6 -
3.3. 电子签章及验章	- 7 -
3.4. 加密解密	- 9 -
4. 交易系统/交易工具对接 CA 互认共享组件说明	- 11 -
4.1. API 调用协议	- 11 -
4.2. SDK 使用说明	- 17 -
5. 交易系统/交易工具对接接口说明	- 20 -
5.1. 共享登录注册	- 20 -
5.2. 签名与验签	- 23 -
5.3. 电子签章与验章	- 27 -
5.4. 加密解密	- 30 -

1.总体架构图



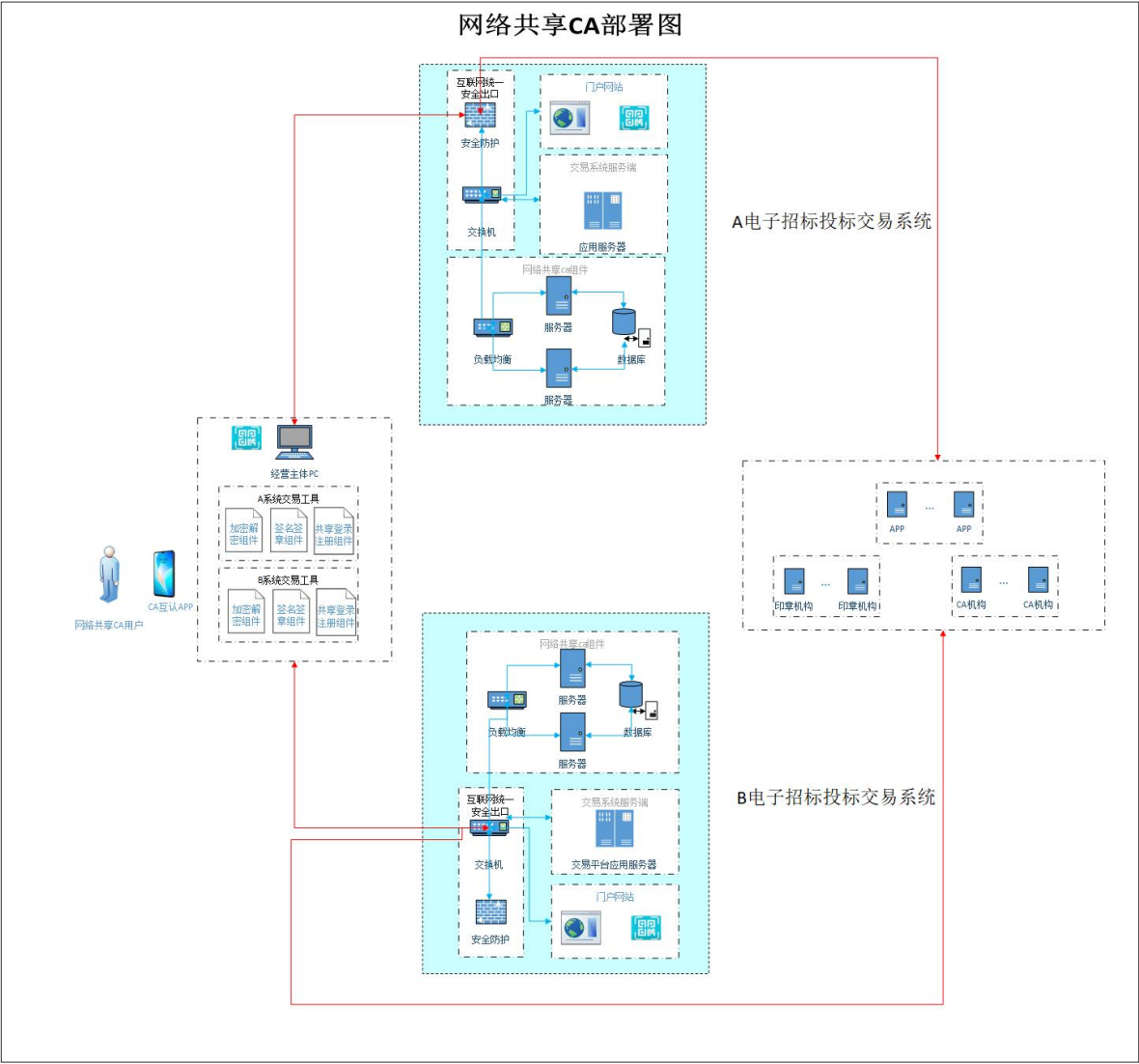
1.1. 软件部署及运维

全国电子招标投标公共服务平台网络体系由中国招标投标公共服务平台（以下简称“国家公共服务平台”）负责组织建设、运行监测和维护，负责依据本标准发布 CA 互认共享组件，承担

该组件及其基础软件在资源服务器中的部署与维护、升级发版工作和漏洞修补工作，并公开 CA 互认共享组件源代码，供各主体免费使用。

在运行过程中，国家公共服务平台可提供必要的指导培训。

1.2. 部署架构图



2.CA 互认共享组件部署

2.1. 硬件及环境准备

电子招标投标交易系统/交易工具运营机构作为移动 CA 数字证书互认推广使用的责任主体，负责提供本地 CA 互认共享组件及其基础软件安装部署所需的资源服务器，并负责资源服务器的安全运行管理。

硬件配置要求如下：

序号	名称	类型	配置	数量	用途
1	负载均衡服务器	实体物理机或虚拟机等 (操作系统 centos7.6-centos7.9 或中标麒麟通用服务器操作系统或统信服务器操作系统 V20)	QPS 不低于 3000, 公网带宽不低于 20M	1	用于实现生产环境 CA 互认共享组件的负载均衡, 可根据平台情况复用
2	应用服务器	实体物理机或虚拟机等 (操作系统 centos7.6-centos7.9 或中标麒麟通用服务器操作系统或统信服务器操作系统 V20)	32C64G 硬盘 2T	2	用于部署 CA 互认共享组件, 生产环境使用
3	应用服务器	实体物理机或虚拟机等 (操作系统 centos7.6-centos7.9 或中标麒麟通用服务器操作系统或统信服务器操作系统 V20)	16C32G 硬盘 1T, 公网带宽不低于 10M	1	用于部署 CA 互认共享组件, 测试环境联调测试使用

环境及软件清单如下:

序号	名称	用途	备注
1	CA 互认共享组件	提供使用互认的移动 CA 数字证书共享登录注册、签章、加解密等相关服务, 为 CA 共享服务客户端提供服务支撑。	每台服务器部署 (外网可访问 CA 互认共享组件端口, 平台侧需开放 CA 互认共享组件入网, 出网白名单)
2	JDK1.8	Java 应用运行环境	每台服务器安装
3	数据库	存储查询 CA 互认共享对接业务数据	使用云数据库或在 CA 互认共享组件内安装 MySQL5.7-MySQL8.0 集群或达梦数据库或人大金仓数据库。
4	Tongweb7.0 (东方通中间件)	作为 CA 互认共享组件运行的容器	每台服务器安装 (选用)

3.交易系统/交易工具对接说明

3.1. 共享登录注册

场景: 用户访问交易系统/交易工具, 注册并登录交易系统/交易工具账号开展电子招标投标业务。

说明: 用户通过 CA 互认 APP 扫描交易系统/交易工具业务界面全国共享互认 CA 入口展示的由 CA 互认共享组件生成的全国统一标准登录二维码, 向交易系统/交易工具自主授权共享已实名认证的注册信息, 验证通过后登录交易系统/交易工具并开展电子招投标业务。通过扫码共

享代替用户手动重复录入注册资料，减少用户录入操作。免去在多个交易系统/交易工具重复录入注册信息的麻烦。

登录码展示界面内容应包含提示用户使用 CA 互认 APP 登录。并引导用户查看相关全国 CA 互认共享目录，国家公共服务平台将通过网站公布全国 CA 互认共享目录。

步骤：

用户选择交易系统/交易工具开通的任意一个 CA 互认 APP，扫描登录码并在 CA 互认 APP 授权登录交易系统/交易工具；交易系统/交易工具获取已实名认证的信息，验证当前登录用户在交易系统/交易工具是否已存在。

(1)当登录用户在交易系统/交易工具中不存在

交易系统/交易工具应将用户引导到用户注册页面，并将所需共享注册信息显示在注册页面；

用户确认无误，授权共享后，引导其进入到完善基本信息的页面，由用户进一步完善信息，完成注册流程成为交易系统/交易工具用户。

(2)当登录用户在交易系统/交易工具已存在

为保障数据安全，交易系统/交易工具可根据机构/个人唯一 ID 提示用户绑定本平台账号，每个唯一 ID 绑定一次后进入交易系统/交易工具业务系统。唯一 ID 根据登录用户来源的 CA 互认 APP 和身份信息生成，若用户更换 CA 互认 APP 后共享登录，则需要根据唯一 ID 更换绑定的 CA 互认 APP。



共享注册登录场景入口（示意图）

共享登录

主体信息完善

办理相关业务

主体类型

☐ 投资人/供应商
☐ 代理机构
☐ 采购人/招标人

企业名称

中远恒连有限公司

统一社会信用代码

4758754394823984

单位类型

国有企业

对公银行卡号

6120384638403628349

法定代表人姓名

张三

法定代表人证件号

510122199710042939

联系人姓名

张三

联系人手机号

13398762738

联系人身份证号

51012219972839402

☐ 我已认真阅读，并同意 [《注册协议》](#)

确认信息无误，注册并登录

将所需共享注册认证信息显示在注册页面（示意图）

共享登录

主体信息完善

办理相关业务

主体类型

☐ 投资人/供应商
☐ 代理机构
☐ 采购人/招标人

企业名称

中远恒连有限公司

统一社会信用代码

4758754394823984

单位类型

国有企业

对公银行卡号

6120384638403628349

法定代表人姓名

张三

法定代表人证件号

510122199710042939

联系人姓名

张三

联系人手机号

13398762738

联系人身份证号

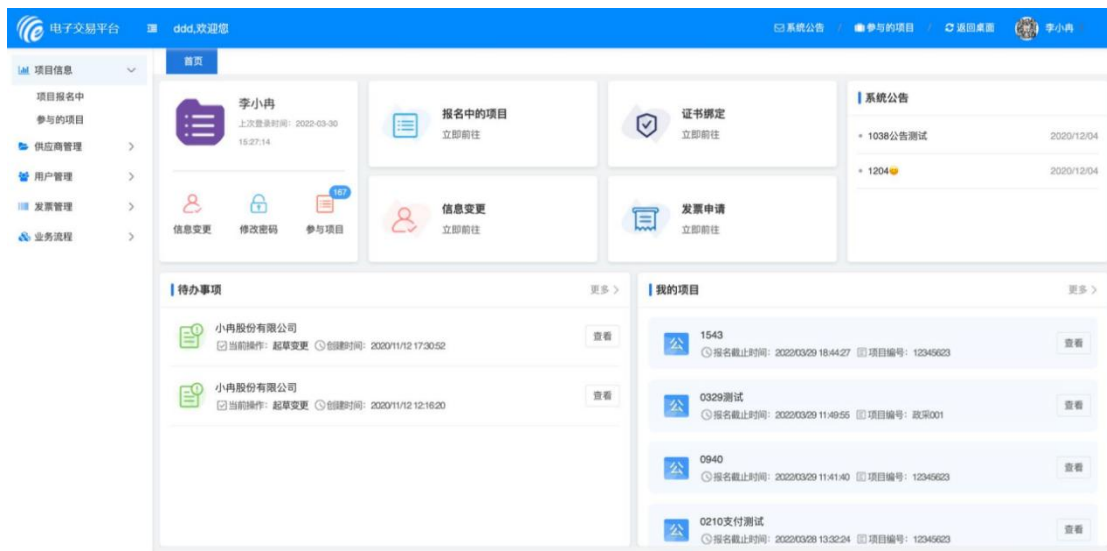
51012219972839402

☐ 我已认真阅读，并同意 [《注册协议》](#)

确认信息无误，注册并登录

注册并登录，再进一步补充完善信息（示意图）

- 5 -



共享注册登录成功（示意图）

3.2. 签名验签

3.2.1. 消息签名

场景：招标方对澄清和答疑文件进行签名。对开标记录等文件进行签名，确保开标过程的透明性和公开性。

说明：对待签名消息进行电子签名，待签名消息可以是文本、文档等内容。电子签名中应包含签章者的 CA 数字证书信息, 从而保证签章者的真实身份，防止身份伪造和冒名签章。

步骤：

(1) 准备要签名的消息

交易系统/交易工具准备好需要进行数字签名的消息。

(2) 提供签章入口：

交易系统/交易工具在消息签名环节提供 CA 互认共享的签名入口。

(3) 选择证书进行签名：

用户选择交易系统/交易工具开通的任意一个 CA 互认 APP。

通过扫描国家标准消息签名码，在 CA 互认 APP 应用程序中选择要使用的 CA 数字证书，授权确认。

(4) 计算消息摘要：

对原始消息进行摘要计算。

常用的摘要算法包括 SM3（国密算法）和 SHA-256。

(5) 生成数字签名：

使用用户 CA 数字证书中的私钥对消息摘要进行加密和签名。

生成的结果即为数字签名，用于验证签名。

(6) 发送签名消息

将生成的数字签名附加到原始消息中，形成一个完整的签名消息。

签名消息通常包括原始消息内容、数字签名以及用户的 CA 数字证书信息。

3.2.2. 消息验签

场景：招标方对澄清和答疑签名文件进行验签。对开标记录等签名文件进行验签。

说明：对已进行过电子签名的消息进行验签，用于检查消息内容是否被篡改或损坏，检查数字签名中的签名者 CA 数字证书，确认签名者的真实身份，防止身份伪造和冒名签名。

步骤：

(1)准备待验证的消息签名

交易系统/交易工具准备包含原始消息、数字签名和 CA 数字证书信息的签名消息。

(2)调用签名验签 SDK 进行验签

调用签名验签 SDK，解析签名消息，从中提取出原始消息、数字签名和 CA 数字证书信息。

签名验签 SDK 从 CA 数字证书中提取出公钥。

签名验签 SDK 使用与签名时相同的摘要算法计算原始消息的摘要。

签名验签 SDK 使用提取的公钥和消息摘要验证数字签名。

(3)获取验签结果

通过以上步骤，验证消息签名的有效性，从而确保消息的完整性和真实性。

3.3. 电子签章及验章

在电子签章业务场景中，电子招标投标交易系统/交易工具应支持用户使用移动 CA 数字证书互认，在相应业务流程环节界面显示 CA 互认共享组件生成的全国统一标准签章二维码。交易系统/交易工具可集成使用随 CA 互认共享组件提供的电子签章工具或组件（代码开源，免费使用）。

交易系统/交易工具通过部署 CA 互认共享组件，前端集成 CA 互认共享电子签章工具/签章组件的方式对接扫码签章、扫码撤章功能，共同完成电子签章。

3.3.1. 文件签章

场景：用户在招投标业务环节，需要对标书、响应文件、合同等电子文档进行单位盖章或个人签名。

说明：对版式文件进行电子签章，国外标准版式文件为 Adobe 的 PDF 文档，我国制定的标准版式文件为 OFD 文档，版式文件是一种固定格式的电子文件，可以精确呈现、打印和分享文档内容。版式文件一经创建，其格式和排版就无法进行二次编辑和篡改。比如传递标书、电子合同、设计图纸等。

步骤：

(1)准备电子文档

用户需要准备好要签章的电子文档，如标书、合同等。

文档格式通常为 PDF、OFD 等常见的版式文件。

(2)提供电子签章入口

交易系统/交易工具在签章环节提供签章入口。通过该入口，交易系统/交易工具可以调起 CA 互认共享电子签章工具/签章组件，用户确定签章，应显示全国统一标准二维码。

(3)选择证书和签章

用户选择交易系统/交易工具开通的任意一个 CA 互认 APP。

用户通过扫描全国标准签章码，在 APP 应用程序中可使用 CA 数字证书进行签章。

(4)计算文件摘要

对待签章文件进行摘要计算。

常用的摘要算法包括 SM3（国密算法）和 SHA-256。

(5)生成数字签名

使用用户 CA 数字证书中的私钥对文件摘要进行加密和签名。

生成的结果即为数字签名，用于验证文件的完整性和签章者身份。

(6)合章并生成签章文件

用户使用印章机构提供的电子印章与数字签名合章。

生成一个带有数字签名和电子印章的新版式文件。

(7)保存已签章文件

将已签章的电子文件保存到安全可靠的位置。

通常保存在用户自己的本地计算机设备中。



文件签章（示意图）

3.3.2. 文件验章

场景：用户完成电子文件签章后，交易系统/交易工具可使用随 CA 互认共享组件提供的电子签章工具或组件对已签电子文件如：合同、响应文件等进行验签。

说明：对已签章的版式文件进行验签，用于检查文件内容是否被篡改或损坏，检查数字签名中的签章者 CA 数字证书,确认签章者的真实身份，防止身份伪造和冒名签章。

步骤：

(1)接收已签章的电子文档

获取已经过数字签名或电子签章的电子文档。

(2)提取数字签名

提取文档中附加的数字签名信息。

(3)获取签章者的 CA 数字证书

从数字签名中提取签章者的 CA 数字证书信息。

CA 数字证书包含签章者的身份信息和公钥等关键数据。

(4)验证 CA 数字证书的有效性

检查 CA 数字证书是否由可信的 CA 机构签发。

验证证书是否在有效期内，是否被吊销或挂失。

确认证书的完整性和真实性。

(5)计算文件摘要值

使用与签章时相同的摘要算法（如 SM3、SHA-256）对电子文档进行摘要计算。

得到当前文件的摘要值。

(6)对比文件摘要值

使用签章者 CA 数字证书中的公钥对数字签名进行解密，得到签章时的文件摘要。

将解密后的摘要与当前文件的摘要进行对比。

如果两个摘要值相同，则说明文件未被篡改，签章有效。

(7)验证签章者身份

根据 CA 数字证书中的身份信息，确认签章者的真实身份。

3.4. 加密解密

交易系统/交易工具在加密环节应支持用户使用移动 CA 数字证书互认，在相应业务流程环节界面显示 CA 互认共享组件生成的全国统一标准加密二维码。交易系统/交易工具可集成使用随 CA 互认共享组件提供的电子加密工具或组件（代码开源，免费使用）。

3.4.1. 文件加密

场景：投标人对投标文件加密。

步骤：

(1)交易系统/交易工具对投标文件加密

交易系统/交易工具前端集成 CA 互认共享加密工具或组件方式对文件进行加密。

交易平台使用对称加密算法（如 sm4）对投标文件进行加密，生成密文文件和随机对称密钥（明文）。

(2)提供加密入口

通过该入口，交易系统/交易工具应调起 CA 互认共享电子加密工具或组件，用户确定加密，应显示全国统一标准加密二维码。

(3)选择证书和签章

用户选择交易系统/交易工具开通的任意一个 CA 互认 APP。

通过扫描全国统一标准加密二维码，在 CA 互认 APP 应用程序中选择要使用的 CA 数字证书，确定加密。

(4)对加密密钥进行非对称加密

对生成的随机对称密钥进行加密和签名，生成加密密钥数字信封。

(5)保存加密密钥数字信封

将加密密钥数字信封保存到安全可靠的位置。



文件加密场景入口（示意图）

3.4.2. 消息加密

交易系统/交易工具可根据业务场景支持用户通过 CA 互认 APP 使用互认的移动 CA 数字证书对任意消息字符串，如报价、文件摘要等不需要形成版式文件的字符串进行加密。

3.4.3. 消息解密

交易系统/交易工具应支持通过网页端和客户端工具集成扫码解密功能，应支持用户通过 CA 互认 APP 使用互认的移动 CA 数字证书实现扫码解密。

文件解密如下图所示：在开标环节添加扫码解密按钮。对消息解密时，在已加密的消息密文旁添加扫码解密按钮。



消息解密场景入口（示意图）

4.交易系统/交易工具对接 CA 互认共享组件说明

4.1.API 调用协议

4.1.1. 概述

CA 互认共享组件的 API 基于 HTTP 协议来调用，开发者可以直接使用中国招标投标公共服务平台提供的 Java-sdk（包含了请求的封装，签名加密，响应解释，性能优化等）来调用，也可以根据 CA 互认共享组件的协议来封装 HTTP 请求进行调用，以下主要是针对自行封装 HTTP 请求进行 API 调用的原理进行详细说明。

4.1.2. 调用入口

调用 CA 互认共享组件 API 的服务 URL 地址。

环境	内容	描述
接口地址	https://CA 互认共享组件 URL 地址/CAShare/Components	CA 互认共享组件 URL, 可以为域名或 ip:port 形式。协议支持 http/https。
请求方式	POST	
请求类型	contentType : 'application/json;charset=UTF-8'	

4.1.3. 设置公共参数

调用任何一个 API 都必须传入的参数，目前支持的公共参数有：

参数名称	类型	是否必填	描述
ServiceCode	String	是	业务模块编码, 具体参见各接口业务模块编码
FeatureCode	String	是	功能接口编码, 具体参见各接口功能接口编码
Version	String	是	CA 互认共享组件服务版本, 统一传 V1.0.0
Authorization	String	是	客户端鉴权信息（规则参见：4.1.5.签名算法）

参数名称	类型	是否必填	描述
Timestamp	String	是	精确到秒的时间戳
Nonce	String	是	请求 ID(随机生成 UUID)
Signature	String	是	对业务数据签名字符串(规则参见：4.1.5.签名算法)
TradingSystemCode	String	是	交易系统编码

4.1.4. 业务参数

API 调用除了必须包含公共参数外，如果 API 本身有业务级的参数也必须传入，每个 API 的业务级参数请参考交易系统/交易工具对接接口说明。

4.1.5. 签名算法

为了防止 API 调用过程中被黑客恶意篡改，调用任何一个 API 都需要携带签名，CA 互认共享组件服务端会根据请求参数，对签名进行验证，签名不合法的请求将会被拒绝。CA 互认共享组件目前支持的签名算法为：HMAC_SM3（sign_method=hmac-sm3），签名大体过程如下：

(1) 引入国密依赖。

pom.xml 里引用国密依赖，代码示例：

```
<dependency>
  <groupId>org.bouncycastle</groupId>
  <artifactId>bcprov-jdk15on</artifactId>
  <version>1.70</version>
</dependency>
```

(2) 获取 clientId、clientSecret，通过移动数字证书（CA）互认自助开通管理平台 CA 互认共享组件信息页面查看并获取。

(3) 构造基础授权 Authorization 信息，并进行 Base64 编码生成规则（"Basic " + base64 加密（clientId + ":" + clientSecret））

代码示例：

```
String authorization = "Basic " + Base64.getEncoder().encodeToString((clientId + ":" + clientSecret).getBytes());
```

(4) 生成 Signature

a. 参数放入 treeMap，内容包括：header 里的（ServiceCode, FeatureCode, Version, Authorization, Timestamp, Nonce, Signature）和业务参数 businessData；特别注意：内容不包括 TradingSystemCode。

b. 将 treeMap 转换为字符串，拼接示例如下：

```
Authorization=Basic
dHJhZGluZ19zeXN0ZW0MzVBRDU5QzUwMUFDJRjVEQ0RGNkYyMjFEODkwOTE3QzM=,FeatureCode=generateLoginQRCode,Nonce=7bd4a245-526a-418d-8a75-3820713cbbd5,ServiceCode=sharedLoginRegistration,Timestamp=1718701688031,Version=V1.0.
```

0,businessData={"signatureData":"12121"}

c.使用 HmacSM3 算法计算签名，生成 Signature：把拼接好的字符串+Timestamp 前三位+Nonce 前三位进行加密。

JAVA 签名示例代码：

```
//主代码块
public String createSignature(String clientId,String clientSecret,String businessData) {
    // 生成请求唯一标识
    String requestUUID = UUID.randomUUID().toString();
    // 获取当前时间戳
    String timestamp = String.valueOf(System.currentTimeMillis());
    // 构造基础授权信息，并进行 Base64 编码
    String authorization = "Basic " + Base64.getEncoder().encodeToString((clientId + ":" +
    clientSecret).getBytes());

    // 创建一个 TreeMap 来有序地存储签名所需的参数
    TreeMap<String,Object> treeMap= new TreeMap<>();
    //a.参数放入 treeMap,内容包括：header 里的（serviceCode，featureCode，Version，
    Authorization，Timestamp，Nonce）和业务参数 businessData
    // 添加业务数据、时间戳、唯一标识、授权信息等参数
    treeMap.put("ServiceCode","sharedLoginRegistration");
    treeMap.put("FeatureCode","generateLoginQRCode");
    treeMap.put("Version","V1.0.0");
    treeMap.put("Timestamp", timestamp);
    treeMap.put("Nonce", requestUUID);
    treeMap.put("Authorization", authorization);
    treeMap.put("businessData", "{\\"signatureData\\":\\"12121\\"}");

    // b.将 TreeMap 转换为字符串，用于签名计算
    String toSignStr = treeMapToString(map);
    // 构造签名盐字符串
    String salt = timestamp.substring(0, 3) + requestUUID.substring(0, 3);
    //accessSecret 和 CA 互认共享组件 accessSecret 一致，默认值：
    256b00c7f8264a9fa1c11e1878f7879a
    String accessSecret="256b00c7f8264a9fa1c11e1878f7879a";
    // c.使用 HmacSM3 算法计算签名
    String signature = HmacSM3Signer.signString(toSignStr + salt,accessSecret);

    return signature;
```

```

}

/**
 * 按照逻辑拼接参数
 *
 * @param map
 * @return
 */

public static String treeMapToString(TreeMap<String, Object> map) {
    StringBuilder stringBuilder = new StringBuilder();
    for (Map.Entry<String, Object> entry : map.entrySet()) {
        // 检查键和值是否都不为 null
        if (entry.getValue() != null) {
            if (stringBuilder.length() > 0) {
                stringBuilder.append(",");
            }
            stringBuilder.append(entry.getKey()).append("=").append(entry.getValue());
        }
    }

    return stringBuilder.toString();
}

//HmacSM3Signer 类如下:
package com.cebpublishservice.common.security.signture;

import org.bouncycastle.crypto.digests.SM3Digest;
import org.bouncycastle.crypto.macs.HMac;
import org.bouncycastle.crypto.params.KeyParameter;

import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;
import java.io.UnsupportedEncodingException;

public class HmacSM3Signer {

    public static final String ENCODING = "UTF-8";

```

```

private static final String ALGORITHM_NAME = "HMAC-SM3";
private static String HASH_SM3 = "SM3";

// @Override
public static String signString(String stringToSign, String signatureSecret) {
    try {
        SecretKey key = new SecretKeySpec((signatureSecret).getBytes(ENCODING),
ALGORITHM_NAME);
        HMac mac = new HMac(new SM3Digest());
        byte[] sign = new byte[mac.getMacSize()];
        byte[] inputBytes = stringToSign.getBytes(ENCODING);
        mac.init(new KeyParameter(key.getEncoded()));
        mac.update(inputBytes, 0, inputBytes.length);
        mac.doFinal(sign, 0);
        return hexEncode(sign);
    } catch (UnsupportedEncodingException e) {
        throw new IllegalArgumentException(e.toString());
    }
}

public static String hexEncode(byte[] raw) {
    if (raw == null) {
        return null;
    }
    StringBuffer sb = new StringBuffer();
    for (int i = 0; i < raw.length; i++) {
        String hex = Integer.toHexString(raw[i] & 0xFF);
        if (hex.length() < 2) {
            sb.append(0);
        }
        sb.append(hex);
    }
    return sb.toString();
}

}

}

public static String hexEncode(byte[] raw) {
    if (raw == null) {
        return null;
    }
    StringBuffer sb = new StringBuffer();
    for (int i = 0; i < raw.length; i++) {

```

```

        String hex = Integer.toHexString(raw[i] & 0xFF);
        if (hex.length() < 2) {
            sb.append(0);
        }
        sb.append(hex);
    }
    return sb.toString();
}
}

```

4. 1. 6. 调用示例

以共享登录注册登录业务场景下生成登录码的功能接口调用为例，具体步骤如下：

```

/**
 *businessData 请求参数拼成的 json 数据
 */
OkHttpClient client = new OkHttpClient().newBuilder()
    .build();
MediaType mediaType = MediaType.parse("text/plain");
RequestBody body = new MultipartBody.Builder().setType(MultipartBody.FORM)
    .addFormDataPart("businessData", "{\\\"signatureData\\\":\\\"签名原文值\\\"}")
    .build();

Request request = new Request.Builder()
    .url("http://localhost:82/CAShare/Components")
    .method("POST", body)
    .addHeader("ServiceCode", "sharedLoginRegistration")
    .addHeader("FeatureCode", "generateLoginQRCode")
    .addHeader("Version", "V1.0.0")
    .addHeader("Authorization", "Basic
dHJhZGluZ19zeXN0ZW06MzVBRDU5QzUwMUFDRjVEQ0RGNkYyMjFEODkwOTE3QzM=")
    .addHeader("Timestamp", "1718701688031")
    .addHeader("Nonce", "7bd4a245-526a-418d-8a75-3820713cbbd5")
    .addHeader("TradingSystemCode", "A123456")
    .addHeader("Signature", "8c173360767173587995686a2273994fcde3748fd7140f9901fbcf10e5677b11")
    .build();
Response response = client.newCall(request).execute();
(1)设置参数值
a.设置公共参数

```

参数名称	类型	是否必填	描述
ServiceCode	String	是	业务模块编码, 具体参见各接口业务模块编码
FeatureCode	String	是	功能接口编码, 具体参见各接口功能接口编码
Version	String	是	CA 互认共享组件服务版本, 统一传 V1.0.0
Authorization	String	是	客户端鉴权信息 (规则参见: 4.1.5.签名算法)
Timestamp	String	是	精确到秒的时间戳
Nonce	String	是	请求 ID(随机生成 UUID)
Signature	String	是	对业务数据签名字符串(规则参见: 4.1.5.签名算法)
tradingSystemCode	String	是	系统标识码 (交易系统标识码)

b. 设置业务参数

参数名称	类型	是否必填	描述
signatureData	String	否	签名数据(用户登录过程中需要进行签名的数据内容)

(2) 生成签名

a. 对公共参数和业务参数按照 ASCII 码顺序进行排序

b. 拼接参数名与参数值

使用 HMAC_SM3 生成签名, 具体规则和 JAVA 示例代码参考签名算法章节描述。

(3) 组装 businessData

请求参数拼成 json 格式, 生成登录码参数示例: {"signatureData": "待签名数据"}

(4) 组装 HTTP 请求

通过 POST 发起请求, 如: https://CA 互认共享组件 URL 地址/CAShare/Components

4. 2. SDK 使用说明

4. 2. 1. 概述

CA 互认共享组件 SDK 提供了 API 的请求封装、摘要签名、响应解释、消息监听等功能, 使用 SDK 可以轻松完成 API 的调用、API 结果的获取、消息的实时监听。

4. 2. 2. 环境依赖

(1) Java 运行环境: 采用 JDK1.8 及以上版本, 确保为电子交易系统/交易工具的稳定运行提供坚实的基础。

(2) 网络连通性: 与 CA 互认共享组件服务间需保持网络顺畅, 确保数据交互无阻。

4. 2. 3. 如何下载

CA 互认共享组件 SDK 可以通过引入 maven 依赖进行集成, cebpubservice-CA-CAShare-java-sdk-SharedLoginRegistration 共享登录注册依赖; cebpubservice-CA-CAShare-java-sdk-SignatureAndVerifySignature 签名与验签依赖; cebpubservice-CA-CAShare-java-sdk-EncryptAndDecrypt 加密与解密依赖, 根据自己业务需要引入相应的业务 SDK, cebpubservice-java-sdk-Core 核心包依赖, 所有业务必须引入该依赖, 引入方式如下:

```

<!-- 共享登录注册依赖 -->
<dependency>
    <groupId>com.cebpublishservice.cashare</groupId>
    <artifactId>cebpublishservice-CA-CAShare-java-sdk-
SharedLoginRegistration</artifactId>
    <version>1.0.0</version>
</dependency>
<!-- 签名与验签依赖 -->
<dependency>
    <groupId> com.cebpublishservice.cashare </groupId>
    <artifactId>cebpublishservice-CA-CAShare-java-sdk-
SignatureAndVerifySignature</artifactId>
    <version>1.0.0</version>
</dependency>
<!-- 加密与解密依赖 -->
<dependency>
    <groupId> com.cebpublishservice.cashare </groupId>
    <artifactId>cebpublishservice-CA-CAShare-java-sdk-
EncryptAndDecrypt</artifactId>
    <version>1.0.0</version>
</dependency>
<!-- 核心包依赖 -->
<dependency>
    <groupId>c com.cebpublishservice.cashare </groupId>
    <artifactId>cebpublishservice-java-sdk-Core</artifactId>
    <version>1.0.0</version>
</dependency>

```

4.2.4. 服务地址

注：服务地址主要用于请求 API 时填写的 URL 地址。

环境	内容	描述
接口地址	https://CA 互认共享组件 URL 地址 /CAShare/Components	CA 互认共享组件 URL, 可以为域名或 ip:port 形式。协议支持 http/https。

4.2.5. 使用示例

Java 示例代码（以生成登录码为例）：

```

import
com.cebpublishservice.cashare.sdk.sharedloginregistration.SharedLoginRegistrationSingleton;
import
com.cebpublishservice.cashare.sdk.signatureandverifysignature.SignatureAndVerifySignatureSingleto

```

```

n;
import com.cebpublishservice.cashare.sdk.encryptanddecrypt.EncryptAndDecryptSingleton;
import com.cebpublishservice.cashare.sdk.sharedloginregistration.SharedLoginRegistration;
import
com.cebpublishservice.cashare.sdk.sharedloginregistration.param.GenerateLoginQRCodeParamDTO;
import
com.cebpublishservice.cashare.sdk.sharedloginregistration.result.GenerateLoginQRCodeResultDTO;
import com.cebpublishservice.exceptions.ClientException;

public class Config {

    public static void main(String[] args) throws ClientException {
        //初始化参数
        // tradingSystemCode 交易系统编码
        //serviceUrl ca 共享组件服务器地址
        // clientId、clientSecret，通过移动数字证书（CA）互认自助开通管理平台 CA 互认共享
        组件信息页面查看并获取。
        //accessSecret 和 CA 互认共享组件 signatureSecret 一致，具体值可在自助开通里查询
        //初始化共享登录注册
        SharedLoginRegistrationSingleton.init("tradingSystemCode", "serviceUrl", "clientId",
"clientSecret", "signatureSecret");
        //初始化签名与验签
        SignatureAndVerifySignatureSingleton.init("tradingSystemCode", "serviceUrl", "clientId",
"clientSecret", "signatureSecret");
        //初始化加密与解密
        EncryptAndDecryptSingleton.init("tradingSystemCode", "serviceUrl", "clientId",
"clientSecret", "signatureSecret");

        GenerateLoginQRCodeParamDTO generateLoginQRCodeParamDTO=new
GenerateLoginQRCodeParamDTO();
        generateLoginQRCodeParamDTO.setSignatureData("签名原文");
        //调用生成登录码
        GenerateLoginQRCodeResultDTO
generateLoginQRCodeResultDTO=SharedLoginRegistration.generateLoginQRCode(generateLoginQ
RCodeParamDTO);
    }

}

```

5.交易系统/交易工具对接接口说明

5.1. 共享登录注册

5.1.1. 生成登录码

接口说明:

该接口用于按照数据规范生成 CA 互认共享全国统一标准登录二维码。

业务模块编码:

sharedLoginRegistration

功能接口编码:

generateLoginQRCode

请求参数:

参数名称	类型	是否必填	描述
signatureData	String	否	签名数据(用户登录过程中需要进行签名的数据内容)

响应参数:

参数名称	类型	描述
tid	String	事件 ID(格式:平台标识码+UUID)
qrCodeImageBase64	String	二维码 BASE64 图片
qrCodeShowText	String	二维码文本(二维码下方显示内容的跳转地址和文本)
qrCodeJumpUrl	String	二维码文本跳转 URL(二维码下方显示内容的跳转地址)
hyperLinkText	String	二维码跳 URL 超链接(二维码下显示内容的文本需要下划线“_”拆分开后分段显示)

响应示例:

```
{
  "code": 200,
  "success": true,
  "data": {
    "tid": "L150100541298e038c45c684523b8b73a584572b00a",
    "qrCodeImageBase64": "iVBORw0KGgoAAAAN...",
    "qrCodeShowText": "请使用_本平台开通的 APP_扫码",
    "qrCodeJumpUrl":
"http://www.cebpubservice.com/cashareservice/#/appcontentsweb/?tradingSystemCode=L1501005412",
    "hyperLinkText": "请使用 <a
href='http://www.cebpubservice.com/cashareservice/#/appcontentsweb/?tradingSystemCod
e=L1501005412' style='color:#4189EB' target='_blank'>本平台开通的 APP</a> 扫码"
  },
  "msg": ""
}
```

```
}
```

5.1.2. 查询登录码扫码状态

接口说明：

该接口用于查询登录码的扫码状态。

业务模块编码：

sharedLoginRegistration

功能接口编码：

queryQRCodeScannedStatus

请求参数：

参数名称	类型	是否必填	描述
tid	String	是	事件 ID(格式:平台标识码+UUID)

响应参数：

参数名称	类型	描述
tid	String	事件 ID(格式:平台标识码+UUID)
scannedStatus	String	扫码状态(00:未扫码 01:已扫码未授权 02 已失效 03 已扫码已授权)
qrCodeType	String	二维码类型 (06 登录)

响应示例：

```
Json
{
  "code": 200,
  "success": true,
  "data": {
    "tid": "L150100541298e038c45c684555b8b73a584572b0fd",
    "scannedStatus": "01",
    "qrCodeType": "06"
  },
  "msg": ""
}
```

5.1.3. 获取用户认证信息

接口说明：

该接口用于获取用户授权认证信息。

业务模块编码：

sharedLoginRegistration

功能接口编码：

getUserAuthInfo

请求参数：

参数名称	类型	是否必填	描述
tid	String	是	事件 ID(格式: 平台标识码+UUID)

响应参数:

参数名称	类型	描述
tid	String	事件 ID(格式: 平台标识码+UUID)
orgTransactionCode	String	单位唯一 ID(单位来自于每个 APP 的唯一标识, 交易系统/交易工具可以根据此标识对登录的单位进行平台已有账户绑定)
personalTransactionCode	String	个人唯一 ID(个人来自于每个 APP 的唯一标识, 交易系统/交易工具可以根据此标识对登录的个人进行平台已有账户绑定)
idCardHash	String	证件号哈希值
orgCode	String	单位代码
orgName	String	单位名称
loginUserType	String	登录用户类型(01:个人 02:单位)
personalName	String	用户姓名
signatureCert	String	签名证书
publicKeyAlgorithm	String	公钥算法(00:SM2 01:RSA)
telephoneNumber	String	手机号码
idCardType	String	证件类型(01:身份证 99:其他)
orgLicenceType	String	单位证件照类型(97:统一社会信用代码 95:其他)
caOrgType	String	CA 机构编码, 详见: CA 机构编码附录
signatureCertSn	String	签名证书序列号
dataSignatureValue	String	数据签名值(对待签名数据进行数字签名后的值)
signatureData	String	签名数据(用户登录过程中需要进行签名的数据内容)

响应示例:

```

Json
{
  "code": 200,
  "success": true,
  "data": {
    "tid": "E4420000291cd5d8ce84bab42f3a5599561a998d684",
    "orgTransactionCode": "ADB046D61BA44562A9B1184E99718E02",
    "personalTransactionCode": "5B82E2727BC847D8B2E5070C6E1FA149",
    "idCardHash": "166445eae.....",
    "orgCode": "92230382MA1C6PE429",
    "orgName": "测试单位",
  }
}

```

```

    "loginUserType": "02",
    "personalName": "张三",
    "signatureCert": "MIIElzCCAxegAwI.....",
    "publicKeyAlgorithm": "01",
    "telephoneNumber": "15565370123",
    "idCardType": "01",
    "orgLicenceType": "97",
    "caOrgType": "010001",
    "signatureCertSn": "33000000010208628",
    "dataSignatureValue": "MIIgJwYJKoZIhvc.....",
    "signatureData": "cd5d8ce84bab42f3a5599561a998d684"
  },
  "msg": ""
}

```

5.2. 签名与验签

5.2.1. 消息签名

5.2.1.1. 生成消息签名码

接口说明：

该接口用于按照数据规范生成 CA 互认共享全国统一标准消息签名二维码。

业务模块编码：

messageSignature

功能接口编码：

generateMessageSignatureQRCode

请求参数：

参数名称	类型	是否必填	值示例	描述
signatureMessage	String	是	4as65d46awe7 9	待签名消息（即需要进行签名的消息内容）

响应参数：

参数名称	类型	描述
tid	String	事件 ID（格式：平台标识码+UUID）
qrCodeImageBase64	String	二维码 BASE64 图片
qrCodeShowText	String	二维码 BASE64 图片
qrCodeJumpUrl	String	二维码文本(二维码下方显示内容的跳转地址和文本)
hyperLinkText	String	二维码文本跳转 URL(二维码下方显示内容的跳转地址)

响应示例：

```

{

```

```

"code": 200,
"success": true,
"data": {
  "tid": "L1501005412805874c096f84573b5bd4a78e48e63ba",
  "qrCodeImageBase64": "iVBORw0KGgoAAAANSUhEUgAAAZAA...",
  "qrCodeShowText": "请使用_本平台开通的 APP_扫码",
  "qrCodeJumpUrl":
"http://www.cebpubservice.com/cashareservice/#/appcontentsweb/?tradingSystemCode=L1501005412",
  "hyperLinkText": "请使用 <a
href='http://www.cebpubservice.com/cashareservice/#/appcontentsweb/?tradingSystemCod
e=L1501005412' style='color:#4189EB' target='_blank'>本平台开通的 APP</a> 扫码"
},
"msg": ""
}

```

5.2.1.2. 查询消息签名码扫码状态

接口说明：

该接口用于查询消息签名码扫码状态。

业务模块编码：

messageSignature

功能接口编码：

queryQRCodeScannedStatus

请求参数：

参数名称	类型	是否必填	描述
tid	String	是	平台标识码+UUID

响应参数：

参数名称	类型	描述
tid	String	事件 ID（格式：平台标识码+UUID）
scannedStatus	String	扫码状态(00:未扫码 01:已扫码未授权 02 已失效 03 已扫码已授权)
qrCodeType	String	二维码类型（20 消息签名）

响应示例：

```

Json
{
  "code": 200,
  "success": true,
  "data": {
    "tid": "L1501005412805874c096f84573b5bd4a78e48e63ba",

```

```
"scannedStatus": "01",  
"qrCodeType": "20"  
},  
"msg": ""  
}
```

5.2.1.3. 获取消息签名值

接口说明:

该接口用于获取待签名消息的签名值。

业务模块编码:

messageSignature

功能接口编码:

getMessageSignatureValue

请求参数:

参数名称	类型	是否必填	描述
tid	String	是	事件 ID(格式: 平台标识码+UUID)

响应参数:

参数名称	类型	描述
tid	String	事件 ID(格式: 平台标识码+UUID)
messageSignatureValue	String	消息签名值(即对待签名消息签名后的值, p7 格式)
signatureMessage	String	待签名消息(即需要进行签名的消息内容)
signatureCertSn	String	签名证书序列号
signatureCert	String	签名证书
publicKeyAlgorithm	String	公钥算法(00:SM2 01:RSA)
caOrgType	String	CA 机构编码, 详见: CA 机构编码附录
signatureUserType	String	签名用户类型(01:个人 02:单位)
tid	String	事件 ID(格式: 平台标识码+UUID)

响应示例:

```
Json  
{  
  "code": 200,  
  "success": true,  
  "data": {  
    "tid": "L1501005412cd241343ce1549d2b360a71b9095c560",  
    "messageSignatureValue": "MIIGHgYJKoZlhcN....",  
    "signatureMessage": "kMj4pwnSijheGSVZoBRvKXtfQuY=",
```

```

    "signatureCertSn": "3300000010143709",
    "signatureCert": "MIIEKjCCAxKgAwIBAg....",
    "publicKeyAlgorithm": "01",
    "caOrgType": "010001",
    "signatureUserType": "02"
  },
  "msg": ""
}

```

5.2.1.4. 签名值验签

接口说明：

该接口用于对消息签名值进验签。

业务模块编码：

messageSignature

功能接口编码：

verifySignatureValue

请求参数：

参数名称	类型	是否必填	描述
signatureMessage	String	是	待签名消息（即需要进行签名的消息内容）
messageSignatureValue	String	是	消息签名值（即对待签名消息签名后的值）
publicKeyAlgorithm	String	是	公钥算法（00： SM2 01： RSA）

响应参数：

参数名称	类型	是否必填	描述
verifySignatureResult	boolean	是	验签结果(true:验签成功 false:验签失败)

响应示例：

```

Json
{
  "code": 200,
  "success": true,
  "data": {
    "verifySignatureResult": true
  },
  "msg": ""
}

```

5.3. 电子签章与验章

5.3.1. 文件电子签章工具

5.3.1.1.CA 互认共享签章工具

获取随 CA 互认共享组件提供的 CA 互认共享签章工具安装包，根据向导在本地电脑上安装。

安装时，CA 互认共享签章工具安装包会检测运行环境，当运行环境不满足需要时，安装过程中会提示用户下载并安装相应的运行环境包。

注意：工具安装启动前，要完全退出 360 安全卫士、腾讯电脑管家等安全防护软件，确保软件安装过程中注册表键值写入不会被拦截。

5.3.1.2.调起 CA 互认共享签章工具

交易系统/交易工具在签章环节提供展示 CA 互认共享签章入口，通过该入口调起 CA 互认共享签章工具。

调起时要注意：当文件当前处于关闭状态时，可直接调起 CA 互认共享签章工具；当文件当前处于打开状态，要确保文件采用只读模式打开。

交易系统/交易工具调起 CA 互认共享签章工具使用的主要参数：

参数名	类型	必填	说明
registerClassName	String	是	预定的注册表键名，键名为 zzAppSignUtility，工具包安装时向注册表路径为\HKEY_CLASSES_ROOT\{ zzAppSignUtility}的键中写入工具的安裝信息(软件安装路径等)。
serviceUrl	String	是	CA 互认共享组件服务地址
TradingSystemCode	String	是	平台标识码
controlCode	String	是	控件编码(0100S01:cfca)
callbackrequest	String	否	签章工具回调接口(用于调用并返回文件签章状态和页码): 1.接口请求类型: POST 2.BodyJosn 请求参数: { [{filename:"文件名", "page":20", issign:"true"}, {.....},] } 3.返回参数: { "code": "200", "msg": "成功" }

参数名	类型	必填	说明
srcPath	String	是	1.签章原文件路径，即导入工具的待签章文件路径。 2.原文件路径可以是本地路径，也可以是 http(s)文件下载地址。 3.多 Pdf 文件签章时，需要把 Pdf 文件压缩成 Zip 包，并以 http(s)文件下载的方式加载。
destPath	String	是	1.签章后文件的保存路径，即工具签章后导出的路径，交易系统/交易工具从本路径获取到签章结果。 2.保存路径可以是本地路径，也可以是 http(s)文件上传接口的请求地址。若该参数传入值为空时，默认启用文件预览模式。 3.如果 Pdf 文档以下载 Zip 压缩包的方式提供。将签章完成后的 Pdf 文件及证书信封(xml)文件合并打成 Zip 压缩包，以 http(s)请求方式上传。
openPage	String	否	打开文件第几页

根据交易系统/交易工具技术架构不同，调起方式分为两种：

(1) 当交易系统/交易工具是客户端时

```
//注册表路径\HKEY_CLASSES_ROOT\{registerClassName}中读工具的启动路径
RegistryKey regSign=
Registry.ClassesRoot.OpenSubKey(registerClassName+"\\shell\\open\\command", true);
if (regSign== null)
{
    //CA 共享互认签章工具未安装，提示安装工具
}
else
{
    //CA 共享互认签章工具程序启动路径
    string instrPath = regSign.GetValue("").ToString().Replace("%1", "").Trim();
    //创建签章进程
    Process process1 = new Process();
    process1.StartInfo.FileName = instrPath;
    //传参使用 json 转 base64 形式
    string jsonString = "
{"serviceUrl\":\"http://123.56.236.113:8868/\", \"tradingSystemCode\":\"A3401314520\", \"
controlCode\":\"0100S01\", \"callbackrequest\":\"http://123.56.236.113:8868/callback/\", \"
destPath\":\"D:\\002.pdf\", \"srcPath\":\"D:\\001.pdf\", \"openPage\":\"1\"}";
    string jsonBase64 = Convert.ToBase64String(Encoding.UTF8.GetBytes(jsonString));
    process1.StartInfo.Arguments = jsonBase64;
    //开始签章，等待 CA 共享互认签章工具关闭后返回
```

```
process1.Start();
process1.WaitForExit();
}
```

(2) 当交易系统/交易工具是网页时

//从网页前端调起 CA 共享互认签章工具触发签章时，执行以下代码(javascript):

Location.href=registerClassName+"://"+jsonBase64

registerClassName: 协议名称(签章工具: zzAppSignUtility)

jsonBase64(支持多文件): 参数以 Json 字符串的格式进行 Base64 加密。Json 签章工具参数格式如下:

```
{
  "serviceUrl": "http://123.56.236.113:8868/", // CA 互认共享组件服务地址(必填)
  "tradingSystemCode": "A3401314520", // 平台编码(必填)
  "controlCode": "0100S01", // 控件编码(必填)
  "callbackrequest": "http://localhost:9291/images/SignFileCallback",
  // 签章客户端工具关闭时，如果存在 callbackrequest 回调请求，则调用并返回文件签章状态和页码
  "srcPath": "D:\\118.pdf", //源文件地址
  "destPath": "E:\\500.pdf" //目标文件地址
  "openPage": "3" //默认打开文件第几页
}
```

5.3.1.3.签章/撤章和保存返回

CA 互认共享签章工具被调起后，从 srcPath 路径加载签章原文件并展示。点击扫码签章按钮，获取签章二维码。

CA 互认共享签章工具调用 CA 互认共享组件中的 CA 共享签名盖章服务端完成签章功能。

以签章工具为例，示意说明如下:



点击“完成签章”按钮，签章后的文件将保存在 **destPath**(本地路径)或上传至 **destPath**(文件上传接口，该接口交易系统/交易工具提供)，CA 共享互认签章工具将关闭，并返回交易系统/交易工具操作界面。



签章过程中，若需要撤销签章，点击“扫码撤章”，可以撤销用户本人使用该工具签署的电子印章。

5.3.1.4. 交易系统/交易工具获取签章文件

交易系统/交易工具通过 **destPath** 路径获取签章后的文件。

由于 **destPath** 路径可以为服务器上传接口，为防止接口暴露引起安全风险，在调起工具传入上传接口地址时，可以在接口地址中拼接增加 **token** 等校验参数，校验请求是否合法。也可以拼接增加一些具有时效性的参数，签章工具或签章组件在调用上传接口时，服务端可以通过校验这些参数是否在规定的时效内，从而实现上传接口只能在规定的时间内调用，从而达到对接口的安全性要求。

5.3.2. 文件签章组件

参见《网络共享 CA 数字证书技术标准签章组件接入指南_电子招标投标交易系统/交易工具侧》。

5.4. 加密解密

5.4.1. 文件加密工具/组件

电子招标投标交易系统/交易工具可自行选择加密组件或加密工具实现加密。

文件对称加密算法：SM4。

对称密钥非对称加密算法：RSA 或 SM2。

5.4.1.1. 文件加密工具

(1) 安装 CA 互认共享加密工具

获取 随 CA 互认共享组件提供的 CA 互认共享加密工具安装包，根据向导在用户本地电脑上安装。

安装时，CA 互认共享加密工具安装包会检测运行环境，当运行环境不满足需要时，安装过程中会提示用户下载并安装相应的运行环境包。

注意：工具安装启动前，要完全退出 360 安全卫士、腾讯电脑管家等安全防护软件，确保软件安装过程中注册表键值写入不会被拦截。

(2) 调起 CA 互认共享加密工具

交易系统/交易工具在加密环节提供 CA 互认共享加密入口，通过该入口调起 CA 互认共享加密工具。

交易系统/交易工具使用以下参数调起 CA 互认共享加密工具：

参数名	类型	是否必填	说明
registerClassName	String	是	预定的注册表键名，键名为 zzAppEncryptUtility ，工具包安装时向注册表路径为 \\HKEY_CLASSES_ROOT\\{zzAppEncrypt Utility}的键中写入工具的安装信息(软件 安装路径等)。
serviceUrl	String	是	CA 互认共享组件服务地址
tradingSystemCode	String	是	系统标识码
fileList	array	是	需加密的文件路径集合
outFilePath	String	是	1.加密结果 ZIP 文件导出路径，ZIP 文件 中压缩了加密后的文件集合与携带加密 信封信息的 XML 文件。 2.保存路径可以是本地路径，也可以是 http(s)文件上传接口的请求地址。

根据交易系统/交易工具的技术架构不同，调起方式分为两种：

(1) 当交易系统/交易工具是客户端时

```
//注册表路径\\HKEY_CLASSES_ROOT\\{registerClassName}中读工具的启动路径
RegistryKey regEncrypt=
Registry.ClassesRoot.OpenSubKey(registerClassName+"\\shell\\open\\command", true);
if (regEncrypt== null)
{
    //CA 互认共享加密工具未安装，提示安装加密模块
}
else
{
    //CA 互认共享加密工具程序启动路径
    string instrPath = regEncrypt.GetValue("").ToString().Replace("%1\\", "").Trim();
    //创建加密进程
    Process process1 = new Process();
    process1.StartInfo.FileName = instrPath;
    //传参使用 json 转 base64 形式
```

```

string jsonString =
"{\"fileList\": [\"D:\\118.pdf\", \"D:\\500.pdf\"], \"outFilePath\": \"D:\\test.zip\", \"tradingSystemCode\": \"A3401314520\", \"serviceUrl\": \"http://123.56.236.113:8868/\"}";
string jsonBase64 = Convert.ToBase64String(Encoding.UTF8.GetBytes(jsonString));
process1.StartInfo.Arguments = jsonBase64;
//开始加密，等待 CA 互认共享加密工具关闭后返回
process1.Start();
process1.WaitForExit();
}

```

(2) 当交易系统/交易工具是网页时

```

//从网页前端调起 CA 互认共享加密工具触发加密时，执行下码(javascript):
Location.href=registerClassName+":"+jsonBase64
registerClassName: 协议名称(加密工具: zzAppEncryptUtility)
jsonBase64(支持多文件): 参数以 Json 字符串的格式进行 Base64 加密。Json 加密工具
参数格式如下:
{
    "serviceUrl": "http://123.56.236.113:8868/", // CA 互认共享组件服务地址(必填)
    "tradingSystemCode": "A3401314520", // 平台编码(必填)
    "fileList": ["D:\\118.pdf", "D:\\500.pdf"], // 需加密的文件路径集合(必填)
    "outFilePath": "test.zip", // 导出 zip 文件路径
}

```

(3) 加密和导出返回

CA 互认共享加密工具被调起后，加载源文件路径上的待加密文件集合。点击“扫码加密”按钮，获取加密二维码。

点击右上角“一键导出”按钮，加密后的 ZIP 文件包将保存在 outFilePath 路径或通过 outFilePath 上传，CA 互认共享加密工具将关闭，并返回交易系统/交易工具操作界面。

由于 outFilePath 路径可以为服务器上传接口，为防止接口暴露引起安全风险，在调起工具传入上传接口地址时，可以在接口地址中拼接增加 token 等校验参数，校验请求是否合法。也可以拼接增加一些具有时效性的参数，工具在调用上传接口时，服务端可以通过校验这些参数是否在规定的时效内，从而实现上传接口只能在规定的时间内调用，从而达到对接口的安全性要求。

(4) 获取加密后的 ZIP 文件包

交易系统/交易工具从 outFilePath 路径获取加密后的 ZIP 文件包。待解密时可解压 ZIP 文件，解析 XML 文件提取出信封信息用于解密。XML 文件记录的信封信息列表如下：

节点名称	类型	描述
ORG_CODE	String	单位代码
ID_CARD_HASH	String	证件号哈希值
ENCRYPTION_USER_TYPE	String	加密用户类型(01:个人 02:单位)

节点名称	类型	描述
TRADINGSYSTEMCODE	String	系统标识码
ENCRYPTED_KEY_DATA_ENVELOPE_LIST		
└ SYMMETRIC_KEY_ALGORITHM		文件对称加密算法(01:SM4)
└ ENCRYPTED_KEY_DATA_ENVELOPE_ID	String	加密密钥信封唯一标识
└ ENCRYPTION_KEY	String	加密后的密钥(对称密钥的加密密文, 对称密钥用于文件加解密)
└ SOURCE_FILE	String	原文件名
└ ENCRYPTED_FILE	String	加密后的文件名
└ CA_ORG_TYPE	String	CA 机构编码, 详见: CA 机构编码附录
└ ENCRYPTION_CERT_SN	String	加密证书序列号

5.4.1.2.加密组件

加密组件开源免费提供, 在国家公共服务平台移动数字证书(CA)互认专栏下载, 由各个电子招标投标交易系统/交易工具自主集成。

5.4.2.消息加密

5.4.2.1.生成消息加密码

接口说明:

该接口用于按照数据规范生成 CA 互认共享全国统一标准消息加密二维码。

业务模块编码:

messageEncrypt

功能接口编码:

generateMessageEncryptQRCode

请求参数:

无。

响应参数:

参数名称	类型	描述
tid	String	事件 ID(格式:平台标识码+UUID)
qrCodeImageBase64	String	二维码 BASE64 图片
qrCodeShowText	String	二维码文本跳转 URL(二维码下方显示内容的跳转地址)
qrCodeJumpUrl	String	二维码文本跳转 URL(二维码下方显示内容的跳转地址)
hyperLinkText	String	二维码跳 URL 超链接(二维码下显示内容的文本需要下划线“_”拆分开后分段显示)

响应示例:

```

{
  "code": 200,
  "success": true,
  "data": {
    "tid": "L1501005412effde41e02fb4f1da23c23a18ce52bea",
    "qrCodeImageBase64": "iVBORw0KGgoAAAANSUhEUgAAA...",
    "qrCodeShowText": "请使用_本平台开通的 APP_扫码",
    "qrCodeJumpUrl":
"http://www.cebpubservice.com/cashareservice/#/appcontentsweb/?tradingSystemCode=L1501005412",
    "hyperLinkText": "      请      使      用      <a
href='http://www.cebpubservice.com/cashareservice/#/appcontentsweb/?tradingSystemCod
e=L1501005412' style='color:#4189EB' target='_blank'>本平台开通的 APP</a> 扫码"
  },
  "msg": ""
}

```

5.4.2.2.查询消息加密码扫码状态

接口说明:

该接口用于查询消息加密码扫码状态。

业务模块编码:

messageEncrypt

功能接口编码:

queryQRCodeScannedStatus

请求参数:

参数名称	类型	是否必填	描述
tid	String	是	事件 ID(格式:平台标识码+UUID)

响应参数:

参数名称	类型	描述
tid	String	事件 ID(格式:平台标识码+UUID)
scannedStatus	String	扫码状态(00:未扫码 01:已扫码未授权 02 已失效 03 已扫码已授权)
qrCodeType	String	二维码类型 (16 消息加密)

响应示例:

```

Json
{
  "code": 200,
  "success": true,
  "data": {

```

```

        "tid": "L15010054122be17f9b227f477280b3baae81c5192f",
        "scannedStatus": "01",
        "qrCodeType": "16"
    },
    "msg": ""
}

```

5.4.2.3. 获取用户加密证书

接口说明:

该接口用于获取用户加密证书信息，以便进一步使用加解密控件完成加密。

业务模块编码:

messageEncrypt

功能接口编码:

getUserEncryptionCert

请求参数:

参数名称	类型	是否必填	描述
tid	String	是	事件 ID(格式:平台标识码+UUID)

响应参数:

参数名称	类型	描述
tid	String	事件 ID(格式:平台标识码+UUID)
personalName	String	用户姓名
telephoneNumber	String	手机号码
idCardHash	String	证件号哈希值
orgCode	String	单位代码
orgName	String	单位名称
encryptionUserType	String	加密用户类型(01:个人 02:单位)
publicKeyAlgorithm	String	公钥算法(00:SM2 01:RSA)
caOrgType	String	CA 机构编码，详见：CA 机构编码附录
encryptionCertSn	String	加密证书序列号
encryptionCert	String	加密证书
signatureCertSn	String	签名证书序列号
signatureCert	String	签名证书
signatureData	String	待签名数据(即需要进行签名的数据内容)
dataSignatureValue	String	数据签名值(即对数据内容签名后的值)

响应示例:

```

Json
{
  "code": 200,
  "success": true,
  "data": {
    "tid": "L1501005412a467e8f2134d45deb2460e8919f63e94"
    "personalName": "张三",
    "telephoneNumber": "15600250123",
    "idCardHash": "c47fea3b7c8e931b9941412e...",
    "orgCode": "92230382MA1C6PE429",
    "orgName": "测试单位",
    "encryptionUserType": "02",
    "publicKeyAlgorithm": "01",
    "caOrgType": "010001",
    "encryptionCertSn": "3300000009842395",
    "encryptionCert": "MIIEKTCCAxGgAwIBAgIIMwAAAAmE...",
    "signatureCertSn": "3300000009842394",
    "signatureCert": "MIIEKTCCAxGgAwIBAgIIMwAAAAmEI5Q...",
    "signatureData": "9f488208d88b4ac98371e740451e864a",
    "dataSignatureValue": "MIIGIQYJKoZlIhvcNAQcCoIIG..."
  },
  "msg": ""
}

```

5.4.3. 消息解密

5.4.3.1. 生成消息解密码

接口说明：

该接口用于按照数据规范生成 CA 互认共享全国统一标准消息解密二维码。

业务模块编码：

messageDecrypt

功能接口编码：

generateMessageDecryptQRCode

请求参数：

参数名称	类型	是否必填	描述
idCardHash	String	是	证件号哈希值
decryptionUserType	String	是	解密用户类型(01:个人 02:单位)
orgCode	String	是	单位代码

响应参数：

参数名称	类型	描述
tid	String	事件 ID(格式:平台标识码+UUID)
qrCodeImageBase64	String	二维码 BASE64 图片
qrCodeShowText	String	二维码文本跳转 URL(二维码下方显示内容的跳转地址)
qrCodeJumpUrl	String	二维码文本跳转 URL(二维码下方显示内容的跳转地址)
hyperLinkText	String	二维码跳 URL 超链接(二维码下显示内容的文本需要下划线 “_” 拆分开后分段显示)

响应示例:

```
{
  "code": 200,
  "success": true,
  "data": {
    "tid": "L1501005412effde41e02fb4f1da23c23a18ce52bfc",
    "qrCodeImageBase64": "iVBORw0KGgoAAAANSUhEUgAAA...",
    "qrCodeShowText": "请使用_本平台开通的 APP_扫码",
    "qrCodeJumpUrl":
"http://www.cebpublishservice.com/cashareservice/#/appcontentsweb/?tradingSystemCode=L1501005412",
    "hyperLinkText": "      请      使      用      <a
href='http://www.cebpublishservice.com/cashareservice/#/appcontentsweb/?tradingSystemCode=L1501005412' style='color:#4189EB' target='_blank'>本平台开通的 APP</a> 扫码"
  },
  "msg": ""
}
```

5.4.3.2.查询消息解密码扫码状态

接口说明:

该接口用于查询消息解密码扫码状态。

业务模块编码:

messageDecrypt

功能接口编码:

queryQRCodeScannedStatus

请求参数:

参数名称	类型	是否必填	描述
tid	String	是	事件 ID(格式:平台标识码+UUID)

响应参数:

参数名称	类型	描述
tid	String	事件 ID(格式:平台标识码+UUID)

参数名称	类型	描述
scannedStatus	String	扫码状态(00:未扫码 01:已扫码未授权 02 已失效 03 已扫码已授权)
qrCodeType	String	二维码类型（14 消息解密）

响应示例:

```

{
  "code": 200,
  "success": true,
  "data": {
    "tid": "L15010054127b8723ed40024af4856cc7c36a9eec58",
    "scannedStatus": "01",
    "qrCodeType": "14"
  },
  "msg": ""
}

```

5.4.3.3.获取解密用户信息

接口说明:

该接口用于获取解密用户信息。

业务模块编码:

messageDecrypt

功能接口编码:

getDecryptUserInfo

请求参数:

参数名称	类型	是否必填	描述
tid	String	是	事件 ID(格式:平台标识码+UUID)

响应参数:

参数名称	类型	描述
tid	String	事件 ID(格式:平台标识码+UUID)
accessToken	String	授权令牌
decryptionUserType	String	解密用户类型（01:个人 02:单位）
idCardHash	String	证件号哈希值
orgCode	String	单位代码

响应示例:

```

{
  "code": 200,

```

```

"success": true,
"data": {
  "tid": "L15010054126e93938c75904e3687d976cba99f62c7",
  "accessToken": "eyJhbGciOiJIUzI1NiJ9.eyJqdGkiOiJiNGZmZT...",
  "decryptionUserType": "02",
  "idCardHash": "c47fea3b7c8e931b9941...",
  "orgCode": "92230382MA1C6PE429"
},
"msg": ""
}

```

5.4.3.4.上传加密密钥信封

接口说明:

该接口用于上传加密密钥信封。

业务模块编码:

messageDecrypt

功能接口编码:

uploadEncryptedKeyDataEnvelope

请求参数:

参数名称	类型	是否必填	描述
tid	String	是	事件 ID(格式:平台标识码+UUID)
accessToken	String	是	授权令牌
encryptedKeyDataEnvelopeList			
└ encryptedKeyDataEnvelopeId	String	是	加密密钥信封唯一标识(交易系统可自定义生成或使用加密密钥信封文件中的对应参数)
└ caOrgType	String	是	CA 机构编码, 详见: CA 机构编码附录
└ encryptionCertSn	String	是	加密证书序列号
└ encryptionKey	String	是	加密后的密钥(对称密钥的加密密文, 对称密钥用于文件加解密)
└ publicKeyAlgorithm	String	是	公钥算法(00:SM2 01:RSA)

响应示例:

```

Json
{
  "code": 200,
  "success": true,
  "data": null,
  "msg": ""
}

```

5.4.3.5.获取解密后的数据

接口说明:

该接口用于获取解密后的数据。

业务模块编码:

messageDecrypt

功能接口编码:

getDecryptedData

请求参数:

参数名称	类型	是否必填	描述
tid	String	是	事件 ID(格式:平台标识码+UUID)

响应参数:

参数名称	类型	描述
tid	String	事件 ID(格式:平台标识码+UUID)
decryptStatusInfo	String	解密状态描述(若解密失败则根据 decryptStatus 字段返回的状态码解释当前解密失败的原因)
decryptStatus	String	解密状态(解密成功或解密失败, 0:解密成功 -1:解密失败 -2:APP 未找到证书 -3:该标段非当前 App 账号加密,请使用原加密 App 账号进行解密 -4:用户 APP 端取消解密)
decryptedKey	String	解密后的明文对称密钥(明文对称密钥用于文件解密)
encryptedKeyDataEnvelopeId	String	加密密钥信封的唯一标识(交易系统可自定义生成或使用加密密钥信封文件中的对应参数)

响应示例:

```
Json
{
  "code": 200,
  "success": true,
  "data": [{
    "tid": "L1501005412a24f52d6956a4d779dc3e08acd0377a5",
    "decryptStatusInfo": "解密成功",
    "decryptStatus": "0",
    "decryptedKey": "AH-0001",
    "encryptedKeyDataEnvelopeId": "e3bab45183714153a2cd84b23fa3e603",
  }],
  "msg": ""
}
```